

```
"""
```

CSV Loader — MVQueen Engine

Loads, cleans, and prepares product catalogs.

```
"""
```

```
import pandas as pd
```

```
from mvqueen_engine.utils.text_cleaning import (
```

```
    strip_html, normalize_whitespace, enforce_brand
```

```
)
```

```
from mvqueen_engine.metafields.metafield_engine import generate_metafields
```

```
def load_and_optimize_csv(path: str):
```

```
    """Load CSV, clean text, generate metafields, return optimized DataFrame."""
```

```
    df = pd.read_csv(path)
```

```
    # Clean HTML + whitespace
```

```
    if "Body (HTML)" in df.columns:
```

```
        df["Body (HTML)"] = df["Body  
(HTML)"].astype(str).apply(strip_html).apply(normalize_whitespace).apply(enforce_brand)
```

```
    # Generate metafields
```

```
    for idx, row in df.iterrows():
```

```
        handle = row.get("Handle", "")
```

```
        metafields = generate_metafields(handle)
```

```
        for key, value in metafields.items():
```

```
            df.loc[idx, key] = value
```

```
    return df
```

```
"""
```

Bundle Generator — MVQueen Engine

Creates bundle rows based on deterministic logic.

```
"""
```

```
Import pandas as pd
```

```
From mvqueen_engine.utils.deterministic import deterministic_multi
```

```
Def generate_bundles(df):
```

```
    """Create bundles based on deterministic product groupings."""
```

```
    Bundles = []
```

```
    For idx, row in df.iterrows():
```

```
        Handle = row["Handle"]
```

```
        Title = row["Title"]
```

```
        # Pick 2–3 deterministic partner products
```

```
        Partners = deterministic_multi(df["Handle"].tolist(), handle, count=3)
```

```
        Bundle_title = f"{title} — MVQueen Bundle"
```

```
        Bundle_handle = f"{handle}-bundle"
```

```
        Bundles.append({
```

```
            "Handle": bundle_handle,
```

```
    "Title": bundle_title,
    "Body (HTML)": f"<p>A curated MVQueen bundle featuring {', '.join(partners)}.</p>",
    "Tags": "bundle, mvqueen",
})
```

```
Return pd.DataFrame(bundles)
```

```
"""
```

Catalog Processor — MVQueen Engine

Runs Phase 1, Phase 2, Phase 3, and prepares for Shopify sync.

```
"""
```

Import pandas as pd

From mvqueen_engine.catalog_processor.csv_loader import load_and_optimize_csv

From mvqueen_engine.catalog_processor.bundle_generator import generate_bundles

```
Def run_full_pipeline(path: str):
```

```
    """Run all catalog phases and return final DataFrame."""
```

```
    # Phase 1 — Load + optimize
```

```
    Df = load_and_optimize_csv(path)
```

```
    # Phase 2 — Bundles
```

```
    Bundle_df = generate_bundles(df)
```

```
    # Combine
```

```
    Final_df = pd.concat([df, bundle_df], ignore_index=True)
```

Return final_df

Main.py

"""

MVQueen Engine — Main Execution File

Runs the full pipeline: Phase 1, Phase 2, Phase 3, and optional Shopify sync.

"""

Import pandas as pd

From mvqueen_engine.catalog_processor.processor import run_full_pipeline

From mvqueen_engine.shopify_api.product_updater import update_products

Def main():

Input CSV

Input_path = "input.csv" # You will replace this with your actual file

Output_path = "output.csv"

Print("Loading and processing catalog...")

Df = run_full_pipeline(input_path)

Print("Saving optimized catalog...")

Df.to_csv(output_path, index=False, encoding="utf-8")

Print("Catalog saved to:", output_path)

Optional Shopify Sync

```
DO_SHOPIFY_SYNC = False # Set to True when ready
```

```
If DO_SHOPIFY_SYNC:
```

```
    Print("Running Shopify sync...")
```

```
    Update_products(df, dry_run=True)
```

```
    Print("Shopify sync complete.")
```

```
If __name__ == "__main__":
```

```
    Main()
```

```
FASHION_SILHOUETTES = [
```

```
    "A-line", "Bodycon", "Slip", "Sheath", "Fit-and-Flare", "Oversized",
```

```
    "Tailored", "Cropped", "High-Waisted", "Wide-Leg", "Straight-Leg",
```

```
    "Relaxed Fit", "Boxy", "Structured", "Draped", "Asymmetric", "Wrap",
```

```
    "Peplum", "Midi", "Mini", "Maxi", "Off-Shoulder", "One-Shoulder",
```

```
    "Halter", "Racerback", "Bustier", "Corset", "Balloon Sleeve",
```

```
    "Puff Sleeve", "Cap Sleeve", "Longline", "Trench", "Bomber",
```

```
    "Blazer Cut", "Skater", "Column", "Empire Waist", "Tiered",
```

```
    "Pleated", "Ruched", "Gathered", "Panelled", "Cut-Out", "Sculpted",
```

```
    "Contour Fit", "Second-Skin", "Fluid Drape", "Box Pleat"
```

```
]
```

```
FASHION_DETAILS = [
```

```
    "gold hardware", "silver hardware", "contrast stitching", "raw hem",
```

```
    "distressed finish", "embellished trim", "pearl accents", "lace panel",
```

```
    "mesh overlay", "satin lining", "double-breasted buttons", "side slit",
```

```
    "front slit", "back vent", "adjustable straps", "elastic waistband",
```

```
    "hidden zipper", "exposed zipper", "button front", "hook-and-eye closure",
```

“belted waist”, “cinched waist”, “pleated front”, “cargo pockets”,
“patch pockets”, “welt pockets”, “ribbed cuffs”, “ribbed hem”,
“feather trim”, “sequin detail”, “beaded detail”, “embroidered detail”,
“quilted texture”, “padded shoulders”, “structured collar”,
“foldover collar”, “shawl collar”, “notched lapel”, “waterfall drape”,
“tie-front”, “wrap closure”, “ruched sides”, “gathered bust”,
“panel stitching”, “contrast piping”, “sheer sleeves”

]

FASHION_MATERIALS = [

“cotton”, “organic cotton”, “linen”, “silk”, “satin”, “velvet”,
“chiffon”, “mesh”, “lace”, “tulle”, “modal”, “bamboo fiber”,
“cashmere”, “wool”, “merino wool”, “alpaca blend”, “faux fur”,
“faux leather”, “vegan leather”, “denim”, “stretch denim”,
“rib knit”, “jersey knit”, “crepe”, “tweed”, “bouclé”, “suede”,
“poly-blend”, “nylon”, “spandex”, “elastane”, “viscose”, “rayon”,
“microfiber”, “performance fabric”, “athletic mesh”, “airflow knit”,
“buttery-soft knit”, “structured twill”, “sateen”, “organza”

]

FASHION_VIBES = [

“quiet luxury”, “soft glam”, “street luxe”, “editorial chic”,
“runway modern”, “romantic feminine”, “minimalist”, “maximalist”,
“clean girl aesthetic”, “old money”, “new money glam”, “city-ready”,
“vacation mode”, “resort chic”, “evening elegance”, “day-to-night”,
“athleisure luxe”, “soft power”, “bold confidence”, “effortless cool”,
“Parisian chic”, “LA glow”, “NYC edge”, “Y2K revival”, “retro modern”,
“classic timeless”, “sultry night-out”, “CEO energy”, “influencer edit”,

“content creator ready”, “festival glam”, “holiday sparkle”

]

Brand bank

SEO_PRIMARY = [

“luxury skincare”, “clean beauty”, “glowing skin”, “hydrating serum”,

“anti-aging routine”, “brightening formula”, “soft glam makeup”,

“long-wear foundation”, “dewy finish”, “glass skin routine”,

“matte finish makeup”, “sensitive skin safe”, “vegan beauty”,

“cruelty-free skincare”, “editorial fashion”, “runway beauty”,

“quiet luxury outfits”, “street style fashion”, “minimalist wardrobe”,

“capsule collection essentials”

]

SEO_SECONDARY = [

“daily moisturizer”, “skin barrier repair”, “SPF protection”,

“lightweight foundation”, “soft matte look”, “hydrating lip gloss”,

“long-lasting wear”, “buildable coverage”, “natural glow”,

“even skin tone”, “texture smoothing”, “fine line care”,

“clean girl aesthetic”, “soft glam routine”, “night-out makeup”,

“event-ready glam”, “vacation outfits”, “date night looks”,

“elevated basics”, “premium fabrics”

]

CATEGORY_TYPES = [

“Skincare”,

“Makeup”,

“Haircare”,

“Bodycare”,

“Fragrance”,
“Tools”,
“Accessories”,
“Apparel”,
“Jewelry”,
“Loungewear”,
“Activewear”,
“Outerwear”,
“Footwear”,
“Bags”,
“Seasonal”,
“Limited Edition”,

]

PRODUCT_TYPES = [

Skincare

“Cleanser”, “Toner”, “Serum”, “Moisturizer”, “SPF”, “Eye Cream”,
“Mask”, “Exfoliant”, “Treatment”,

Makeup

“Foundation”, “Concealer”, “Blush”, “Bronzer”, “Highlighter”,
“Setting Spray”, “Lip Gloss”, “Lipstick”, “Lip Liner”,
“Mascara”, “Brow Gel”, “Eyeshadow”,

Haircare

“Shampoo”, “Conditioner”, “Hair Mask”, “Hair Oil”, “Styling Cream”,
“Heat Protectant”, “Leave-In Conditioner”,

Bodycare

“Body Lotion”, “Body Oil”, “Body Wash”, “Scrub”,

Fashion

“Dress”, “Top”, “Bottom”, “Jacket”, “Coat”, “Skirt”, “Pants”,

“Shorts”, “Blazer”, “Bodysuit”, “Sweater”, “Hoodie”, “Tee”,

“Tank”, “Corset”, “Lingerie”,

Accessories

“Bag”, “Belt”, “Hat”, “Scarf”, “Jewelry”, “Sunglasses”,

]

TAG_RULES = {

“Skincare”: [“skincare”, “skin”, “clean beauty”, “glow”, “hydration”],

“Makeup”: [“makeup”, “beauty”, “soft glam”, “long wear”],

“Haircare”: [“haircare”, “hair”, “shine”, “repair”],

“Bodycare”: [“bodycare”, “body”, “smooth skin”],

“Fragrance”: [“fragrance”, “perfume”, “scent”],

“Apparel”: [“fashion”, “style”, “outfit”, “wardrobe”],

“Jewelry”: [“jewelry”, “accessories”, “shine”],

“Loungewear”: [“loungewear”, “comfort”, “soft”],

“Activewear”: [“activewear”, “gym”, “performance”],

“Outerwear”: [“outerwear”, “coat”, “jacket”],

}

CATEGORY_KEYWORDS = {

“Skincare”: [

“serum”, “cleanser”, “moisturizer”, “spf”, “sunscreen”, “toner”,
“mask”, “cream”, “gel”, “treatment”, “essence”, “hydrating”

],

“Makeup”: [

“foundation”, “concealer”, “blush”, “bronzer”, “highlighter”,
“lipstick”, “lip gloss”, “mascara”, “eyeshadow”, “liner”, “brow”

],

“Haircare”: [

“shampoo”, “conditioner”, “hair oil”, “hair mask”, “styling”,
“leave-in”, “heat protectant”

],

“Bodycare”: [

“body wash”, “body lotion”, “body oil”, “scrub”, “butter”

],

“Fragrance”: [

“perfume”, “eau”, “fragrance”, “parfum”, “mist”

],

“Apparel”: [

“dress”, “top”, “skirt”, “pants”, “jacket”, “coat”, “hoodie”,
“tee”, “tank”, “corset”, “bodysuit”, “sweater”

],

“Jewelry”: [

“necklace”, “bracelet”, “ring”, “earrings”, “jewelry”

],

“Accessories”: [

“bag”, “belt”, “scarf”, “sunglasses”, “hat”

```
],  
  "Footwear": [  
    "boots", "heels", "sneakers", "sandals"  
  ],  
}  
  
PRODUCT_TYPE_KEYWORDS = {  
  "Serum": ["serum", "ampoule"],  
  "Cleanser": ["cleanser", "wash", "gel cleanser"],  
  "Moisturizer": ["moisturizer", "cream", "lotion"],  
  "SPF": ["spf", "sunscreen", "sun cream"],  
  "Foundation": ["foundation", "skin tint", "base"],  
  "Concealer": ["concealer"],  
  "Blush": ["blush"],  
  "Bronzer": ["bronzer", "contour"],  
  "Highlighter": ["highlighter", "glow stick"],  
  "Lip Gloss": ["lip gloss", "gloss"],  
  "Lipstick": ["lipstick"],  
  "Mascara": ["mascara"],  
  "Eyeshadow": ["eyeshadow", "shadow palette"],  
  "Dress": ["dress"],  
  "Top": ["top", "blouse", "shirt"],  
  "Pants": ["pants", "trousers"],  
  "Skirt": ["skirt"],  
  "Jacket": ["jacket"],  
  "Coat": ["coat"],  
  "Sweater": ["sweater", "knit"],
```

```
    "Hoodie": ["hoodie"],  
    "Bag": ["bag", "tote", "crossbody"],  
}  
"""
```

Category & Product Type Engine — MVQueen

Automatically detects category and product type from title/handle.

```
"""
```

```
From mvqueen_engine.brand_brain.brand_banks import (  
    CATEGORY_KEYWORDS, PRODUCT_TYPE_KEYWORDS  
)
```

```
Def detect_category(title: str):
```

```
    Title_lower = title.lower()
```

```
    For category, keywords in CATEGORY_KEYWORDS.items():
```

```
        If any(k in title_lower for k in keywords):
```

```
            Return category
```

```
    Return "Uncategorized"
```

```
Def detect_product_type(title: str):
```

```
    Title_lower = title.lower()
```

```
    For ptype, keywords in PRODUCT_TYPE_KEYWORDS.items():
```

```
        If any(k in title_lower for k in keywords):
```

```
            Return ptype
```

```
    Return "General""""
```

Tag Engine — MVQueen

Generates tags based on category, product type, benefits, persona, fashion vocab, SEO, and vibes.

“””

```
From mvqueen_engine.brand_brain.brand_banks import (  
    TAG_RULES, FASHION_VIBES, FASHION_MATERIALS, SEO_PRIMARY, SEO_SECONDARY  
)
```

```
From mvqueen_engine.utils.deterministic import deterministic_choice,  
deterministic_multi
```

```
Def generate_tags(row):
```

```
    Tags = set()
```

```
    # Category tags
```

```
    Category = row.get("Category", "")
```

```
    If category in TAG_RULES:
```

```
        Tags.update(TAG_RULES[category])
```

```
    # Product type tag
```

```
    Ptype = row.get("Product Type", "")
```

```
    If ptype:
```

```
        Tags.add(ptype.lower())
```

```
    # Benefit tags
```

```
    Benefits = row.get("metafield.custom.key_benefits", "")
```

```
    For b in benefits.split(","):
```

```
        Tags.add(b.strip().lower())
```

```

# Fashion vibes

Vibe = deterministic_choice(FASHION_VIBES, row["Handle"], 0)

Tags.add(vibe.replace(" ", "-"))


# Materials (if apparel)

If category == "Apparel":

    Material = deterministic_choice(FASHION_MATERIALS, row["Handle"], 1)

    Tags.add(material.replace(" ", "-"))


# SEO tags

Primary = deterministic_choice(SEO_PRIMARY, row["Handle"], 2)

Secondary = deterministic_choice(SEO_SECONDARY, row["Handle"], 3)

Tags.add(primary.replace(" ", "-"))

Tags.add(secondary.replace(" ", "-"))


Return " ".join(sorted(tags))

From mvqueen_engine.catalog_processor.category_engine import detect_category,
detect_product_type

From mvqueen_engine.catalog_processor.tag_engine import generate_tags

# Phase 1.5 — Category + Product Type Detection

Df["Category"] = df["Title"].apply(detect_category)

Df["Product Type"] = df["Title"].apply(detect_product_type)


# Phase 1.6 — Tag Engine

Df["Tags"] = df.apply(generate_tags, axis=1)

```

